

10th Aug, 2026

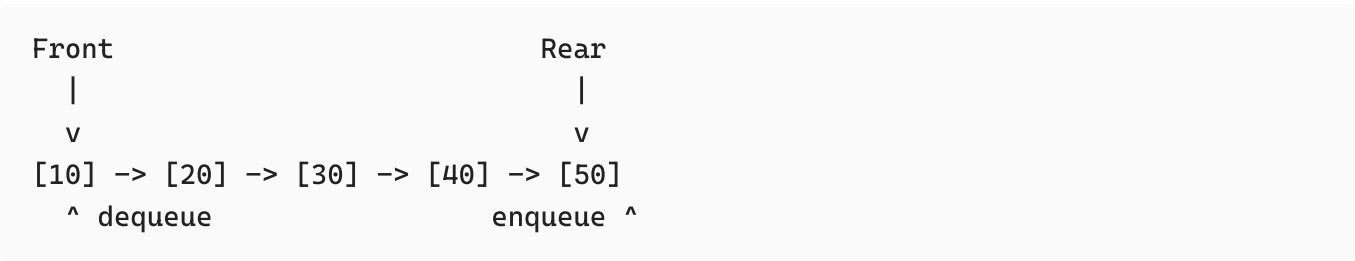
Queue — Data Structures & Algorithms

Comprehensive notes covering theory, all queue variants, and C++ implementations.

1. What is a Queue?

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle — the element inserted first is the one removed first, just like a real-world queue/line.

- **Insertion** happens at the **rear (back/tail)**.
- **Deletion** happens at the **front (head)**.



Core Operations

Operation	Description	Time Complexity (ideal)
<code>enqueue(x)</code>	Insert element <code>x</code> at rear	$O(1)$
<code>dequeue()</code>	Remove element from front	$O(1)$
<code>front()/peek()</code>	Return front element without removing	$O(1)$
<code>rear()</code>	Return rear element without removing	$O(1)$
<code>isEmpty()</code>	Check if queue has no elements	$O(1)$
<code>isFull()</code>	Check if queue is full (bounded queue)	$O(1)$
<code>size()</code>	Number of elements currently in queue	$O(1)$

Real-world Applications

- CPU/OS task scheduling (Round Robin scheduler)
- Handling requests in web servers, print spoolers

- BFS traversal in graphs/trees
 - Buffering data streams (IO buffers, keyboard buffer)
 - Call center / customer service systems
 - Cache implementation (LRU uses deque)
-

2. Ways to Implement a Queue

1. **Array-based (static)** — fixed size, simple, but naive version wastes space after dequeues.
2. **Circular Array** — reuses freed space, solves the "false full" problem of linear arrays.
3. **Linked List** — dynamic size, no wasted memory, $O(1)$ enqueue/dequeue with tail pointer.
4. **Two Stacks** — classic interview construction of a queue using stack semantics.

The choice of underlying structure changes complexity trade-offs:

Implementation	Enqueue	Dequeue	Space	Resizing needed?
Simple array	$O(1)$	$O(1)^*$	Fixed	Yes (wastage)
Circular array	$O(1)$	$O(1)$	Fixed	Yes (when full)
Linked list	$O(1)$	$O(1)$	Dynamic	No
Two stacks (amortized)	$O(1)$	$O(n)/O(1)$ amortized	Dynamic	No

*Simple array dequeue is $O(1)$ time but leads to unusable freed slots unless you shift elements ($O(n)$) or use circular indexing.

3. Types of Queues

#	Type	Insertion allowed at	Deletion allowed at	Order served
1	Simple / Linear Queue	Rear only	Front only	Strict FIFO
2	Circular Queue	Rear only (wraps around)	Front only (wraps around)	Strict FIFO
3	Double-Ended Queue (Deque)	Front and Rear	Front and Rear	Either end, either order

#	Type	Insertion allowed at	Deletion allowed at	Order served
4	Input-Restricted Deque	Rear only	Front and Rear	FIFO-like with flexible exit
5	Output-Restricted Deque	Front and Rear	Front only	Flexible entry, FIFO-like exit
6	Priority Queue	Anywhere (by priority)	Highest/lowest priority end	By priority, not arrival order

Each row above changes what the underlying implementation must support:

- Rows 1–2 only ever touch the front/rear indices in **one direction each** — arrays are enough.
- Rows 3–5 need **both ends touchable**, so a doubly linked list (or a bidirectionally-wrapping circular array) is the natural fit.
- Row 6 abandons position-based access entirely and orders elements by a **key/priority**, so it needs a heap (or another ordered structure) instead of front/rear pointers.

4. Simple (Linear) Queue

Strict FIFO. Insert at rear, delete from front. In a naive array implementation, once `rear` reaches the end of the array, no more insertions are possible even if front slots are empty — this is the classic drawback that circular queues fix.

Array-based Implementation

```
#include <iostream>
using namespace std;

class LinearQueue {
private:
    int* arr;
    int capacity;
    int frontIdx;
    int rearIdx;
    int count;

public:
    LinearQueue(int size) {
        capacity = size;
    }
};
```

```

    arr = new int[capacity];
    frontIdx = 0;
    rearIdx = -1;
    count = 0;
}

~LinearQueue() {
    delete[] arr;
}

bool isEmpty() const { return count == 0; }
bool isFull() const { return count == capacity; }

void enqueue(int value) {
    if (isFull()) {
        cout << "Queue Overflow\n";
        return;
    }
    rearIdx++; // no wraparound -> rear keeps increasing
    arr[rearIdx] = value;
    count++;
}

int dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return -1;
    }
    int value = arr[frontIdx];
    frontIdx++; // freed slot is never reused
    count--;
    return value;
}

int front() const {
    if (isEmpty()) throw runtime_error("Queue is empty");
    return arr[frontIdx];
}

int size() const { return count; }
};

```

Key limitation: `frontIdx` and `rearIdx` only move forward, so array slots before `frontIdx` are permanently wasted — after enough enqueue/dequeue cycles, `isFull()` can be true even though logically there is free space. This motivates the circular queue.

Linked-List Implementation (avoids the wastage problem entirely)

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class LinkedQueue {
private:
    Node* frontPtr;
    Node* rearPtr;
    int count;

public:
    LinkedQueue() : frontPtr(nullptr), rearPtr(nullptr), count(0) {}

    ~LinkedQueue() {
        while (frontPtr) dequeue();
    }

    bool isEmpty() const { return frontPtr == nullptr; }

    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (isEmpty()) {
            frontPtr = rearPtr = newNode;
        } else {
            rearPtr->next = newNode;
            rearPtr = newNode;
        }
        count++;
    }

    int dequeue() {
        if (isEmpty()) {
            cout << "Queue Underflow\n";
            return -1;
        }
        Node* temp = frontPtr;
        int value = temp->data;
        frontPtr = frontPtr->next;
```

```

        if (!frontPtr) rearPtr = nullptr; // queue became empty
        delete temp;
        count--;
        return value;
    }

    int front() const {
        if (isEmpty()) throw runtime_error("Queue is empty");
        return frontPtr->data;
    }

    int size() const { return count; }
};

```

Why linked-list version is better for unbounded use: no upper capacity, no wasted slots, true $O(1)$ enqueue/dequeue always. Trade-off: extra memory per node (pointer overhead) and heap allocation cost per operation, plus poor cache locality vs contiguous arrays.

Bonus: Queue Using Two Stacks

A well-known construction that builds FIFO behavior purely out of LIFO (stack) primitives — common in interviews. `stack1` absorbs incoming elements; `stack2` is only refilled (by popping everything off `stack1`) when it runs dry, which reverses the order back to FIFO.

```

#include <iostream>
using namespace std;

// Minimal linked-list-backed stack (LIFO) used as a building block below.
class Stack {
private:
    struct SNode {
        int data;
        SNode* next;
        SNode(int val, SNode* n) : data(val), next(n) {}
    };
    SNode* top;

public:
    Stack() : top(nullptr) {}

    bool isEmpty() const { return top == nullptr; }

    void push(int value) {
        top = new SNode(value, top);
    }
}

```

```

int pop() {
    SNode* temp = top;
    int value = temp->data;
    top = top->next;
    delete temp;
    return value;
}

int peek() const { return top->data; }
};

class QueueUsingStacks {
private:
    Stack stack1; // for enqueue
    Stack stack2; // for dequeue

public:
    void enqueue(int value) {
        stack1.push(value);
    }

    int dequeue() {
        if (stack2.isEmpty()) {
            if (stack1.isEmpty()) {
                cout << "Queue Underflow\n";
                return -1;
            }
            while (!stack1.isEmpty()) {
                stack2.push(stack1.pop());
            }
        }
        return stack2.pop();
    }

    bool isEmpty() const {
        return stack1.isEmpty() && stack2.isEmpty();
    }
};

```

Complexity: each element is moved from `stack1` to `stack2` at most once, so total work across `n` operations is $O(n)$ — this gives **amortized $O(1)$** dequeue, even though a single dequeue call can occasionally be $O(n)$ when it triggers the transfer.

5. Circular Queue

Solves the array-wastage problem by treating the array as circular — `rear` wraps around to index `0` using modulo arithmetic once it reaches the end, reusing freed slots from the front.

```
        rear wraps here
          |
[ _ , _ , 30, 40, 50 ]
  ^front      ^rear
  (index 2)   (index 4)

after enqueue(60):
[60, _ , 30, 40, 50 ]
  ^front      ^rear (wrapped to index 0)
```

Distinguishing full vs empty

Both conditions can look like `front == rear` in a naive scheme, so implementations typically use one of:

- A `count` variable (used below — simplest, most readable).
- Sacrificing one slot (`isFull` when `(rear+1)%capacity == front`).
- A separate boolean flag.

Array-based Implementation

```
#include <iostream>
using namespace std;

class CircularQueue {
private:
    int* arr;
    int capacity;
    int frontIdx;
    int rearIdx;
    int count;

public:
    CircularQueue(int size) {
        capacity = size;
        arr = new int[capacity];
        frontIdx = 0;
        rearIdx = -1;
        count = 0;
    }
};
```



```

}

~CircularQueue() { delete[] arr; }

bool isEmpty() const { return count == 0; }
bool isFull() const { return count == capacity; }

void enqueue(int value) {
    if (isFull()) {
        cout << "Circular Queue Overflow\n";
        return;
    }
    rearIdx = (rearIdx + 1) % capacity; // wraparound
    arr[rearIdx] = value;
    count++;
}

int dequeue() {
    if (isEmpty()) {
        cout << "Circular Queue Underflow\n";
        return -1;
    }
    int value = arr[frontIdx];
    frontIdx = (frontIdx + 1) % capacity; // wraparound
    count--;
    return value;
}

int front() const {
    if (isEmpty()) throw runtime_error("Queue is empty");
    return arr[frontIdx];
}

int size() const { return count; }
};

```

Complexity: $O(1)$ for all core operations, $O(n)$ space. This is the same underlying structure used for **circular buffers / ring buffers** in streaming and networking code.

Applications: CPU scheduling (Round Robin), traffic light systems, circular buffers in audio/video streaming, memory management (ring buffers).

6. Double-Ended Queue (Deque)

A **Deque** allows insertion and deletion from **both ends** — front and rear. It generalizes both stacks and queues:

- Using only `push_back / pop_back` → behaves like a stack.
- Using `push_back / pop_front` → behaves like a queue.

Operations

`insertFront`, `insertRear`, `deleteFront`, `deleteRear`, `getFront`, `getRear`.

Implementation notes

- Array-based deque needs **circular indexing in both directions** (front can decrement with wraparound too).
- Doubly linked list is a very natural fit since you need $O(1)$ access/modification at both ends.

Doubly Linked List Implementation

```
#include <iostream>
using namespace std;

class DNode {
public:
    int data;
    DNode* prev;
    DNode* next;
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class Deque {
private:
    DNode* frontPtr;
    DNode* rearPtr;
    int count;

public:
    Deque() : frontPtr(nullptr), rearPtr(nullptr), count(0) {}

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }

    void insertFront(int value) {
        DNode* node = new DNode(value);
        if (isEmpty()) {
            frontPtr = rearPtr = node;
        }
    }
}
```

```

    } else {
        node->next = frontPtr;
        frontPtr->prev = node;
        frontPtr = node;
    }
    count++;
}

void insertRear(int value) {
    DNode* node = new DNode(value);
    if (isEmpty()) {
        frontPtr = rearPtr = node;
    } else {
        rearPtr->next = node;
        node->prev = rearPtr;
        rearPtr = node;
    }
    count++;
}

void deleteFront() {
    if (isEmpty()) { cout << "Deque is empty\n"; return; }
    DNode* temp = frontPtr;
    frontPtr = frontPtr->next;
    if (frontPtr) frontPtr->prev = nullptr;
    else rearPtr = nullptr;
    delete temp;
    count--;
}

void deleteRear() {
    if (isEmpty()) { cout << "Deque is empty\n"; return; }
    DNode* temp = rearPtr;
    rearPtr = rearPtr->prev;
    if (rearPtr) rearPtr->next = nullptr;
    else frontPtr = nullptr;
    delete temp;
    count--;
}

int getFront() const {
    if (isEmpty()) throw runtime_error("Deque is empty");
    return frontPtr->data;
}

int getRear() const {

```

```

        if (isEmpty()) throw runtime_error("Deque is empty");
        return rearPtr->data;
    }
};

```

Complexity: All operations $O(1)$. **Applications:** sliding window problems (max/min in subarray via monotonic deque), palindrome checking, undo/redo in editors, work-stealing schedulers.

Circular Array Implementation (alternative)

Uses `frontIdx` decreasing with `(frontIdx - 1 + capacity) % capacity` and `rearIdx` increasing with `(rearIdx + 1) % capacity` — more memory-efficient (no pointer overhead) but has a fixed capacity unless you implement dynamic resizing manually (allocate a bigger array and copy elements over, similar to how growable arrays are implemented internally).

7. Input-Restricted Deque

A deque variant where **insertion is allowed only from one end (rear)**, but **deletion is allowed from both ends**.

- `insertRear` ✓
- `deleteFront` ✓
- `deleteRear` ✓
- `insertFront` ✗ (restricted)

Implementation

Reuse the `Deque` class above but expose a restricted interface:

```

class InputRestrictedDeque : private Deque {
public:
    using Deque::insertRear;
    using Deque::deleteFront;
    using Deque::deleteRear;
    using Deque::getFront;
    using Deque::getRear;
    using Deque::isEmpty;
    using Deque::size;
    // insertFront is intentionally NOT exposed
};

```

This is the standard technique: implement the general deque once, then **restrict the public interface** for specialized variants rather than writing new low-level logic. This mirrors how it's usually taught — input/output-restricted deques are conceptual restrictions on a full deque, not separate data structures.

8. Output-Restricted Deque

The mirror image: **insertion allowed from both ends, deletion allowed only from one end (front)**.

- insertFront 
- insertRear 
- deleteFront 
- deleteRear  (restricted)

```
class OutputRestrictedDeque : private Deque {
public:
    using Deque::insertFront;
    using Deque::insertRear;
    using Deque::deleteFront;
    using Deque::getFront;
    using Deque::getRear;
    using Deque::isEmpty;
    using Deque::size;
    // deleteRear is intentionally NOT exposed
};
```

9. Priority Queue

Elements are served **not by arrival order but by priority**. The highest (or lowest) priority element is dequeued first; equal priorities usually fall back to FIFO or are implementation-defined.

Underlying implementations and how complexity changes

Underlying structure	Insert	Extract-Max/Min	Notes
Unsorted array/list	O(1)	O(n)	Simple but slow extraction

Underlying structure	Insert	Extract-Max/Min	Notes
Sorted array/list	$O(n)$	$O(1)$	Slow insertion
Binary Heap	$O(\log n)$	$O(\log n)$	Standard, best general-purpose choice
Balanced BST	$O(\log n)$	$O(\log n)$	Also supports arbitrary search
Fibonacci Heap	$O(1)$ amortized	$O(\log n)$ amortized	Used in advanced graph algos (Dijkstra)

Binary heap is the standard implementation because it gives good $O(\log n)$ bounds for both insert and extract while being simple and cache-friendly (array-backed complete binary tree).

Max-Heap Based Priority Queue (Array Implementation)

```
#include <iostream>
using namespace std;

class PriorityQueueMaxHeap {
private:
    int* heap;
    int capacity;
    int count;

    void resize() {
        capacity *= 2;
        int* newHeap = new int[capacity];
        for (int i = 0; i < count; i++) newHeap[i] = heap[i];
        delete[] heap;
        heap = newHeap;
    }

    int parent(int i) { return (i - 1) / 2; }
    int leftChild(int i) { return 2 * i + 1; }
    int rightChild(int i) { return 2 * i + 2; }

    void heapifyUp(int i) {
        while (i > 0 && heap[parent(i)] < heap[i]) {
            swap(heap[parent(i)], heap[i]);
            i = parent(i);
        }
    }

    void heapifyDown(int i) {
```

```

        int largest = i;
        int left = leftChild(i);
        int right = rightChild(i);

        if (left < count && heap[left] > heap[largest])
            largest = left;
        if (right < count && heap[right] > heap[largest])
            largest = right;

        if (largest != i) {
            swap(heap[i], heap[largest]);
            heapifyDown(largest);
        }
    }
}

public:
    PriorityQueueMaxHeap(int initialCapacity = 16) {
        capacity = initialCapacity;
        heap = new int[capacity];
        count = 0;
    }

    ~PriorityQueueMaxHeap() { delete[] heap; }

    bool isEmpty() const { return count == 0; }

    void enqueue(int value) { // insert with priority = value
        if (count == capacity) resize();
        heap[count] = value;
        heapifyUp(count);
        count++;
    }

    int dequeue() { // removes highest priority element
        if (isEmpty()) {
            cout << "Priority Queue Underflow\n";
            return -1;
        }
        int top = heap[0];
        heap[0] = heap[count - 1];
        count--;
        if (count > 0) heapifyDown(0);
        return top;
    }

    int peek() const {

```

```

        if (isEmpty()) throw runtime_error("Priority Queue is empty");
        return heap[0];
    }

    int size() const { return count; }
};

```

Why array-backed heap over pointer-based tree: a complete binary tree stored in an array needs no pointers — children/parent are computed via index arithmetic ($2i+1$, $2i+2$, $(i-1)/2$), giving excellent cache locality and no per-node allocation overhead. `resize()` doubles capacity when full, giving amortized $O(1)$ growth (same idea `std::vector` uses internally, implemented here by hand).

Applications: Dijkstra's shortest path, Prim's MST, Huffman coding, task scheduling by priority, event-driven simulation, A* search.

10. Comparison Summary — Which Queue to Use When

Requirement	Best Choice
Strict FIFO, fixed max size known upfront	Circular Queue (array)
Strict FIFO, unknown/unbounded size	Linked-List Queue
Need access/insertion at both ends	Deque
Need only rear insertion but deletion at both ends	Input-Restricted Deque
Need insertion at both ends, deletion only at front	Output-Restricted Deque
Need to always serve "most important" element first	Priority Queue (Binary Heap)
Sliding-window min/max in $O(n)$	Monotonic Deque
Graph shortest-path / MST algorithms	Priority Queue (often Fibonacci Heap in theory, binary heap in practice)

11. Common Pitfalls / Interview Notes

- **Linear queue "false overflow":** always mention circular queue as the fix when discussing array-based linear queue drawbacks.
- **Circular queue full vs empty ambiguity:** always clarify which technique you're using (`count` variable is cleanest to explain and code under pressure).
- **Queue via two stacks:** classic problem — push to `stack1` for enqueue; for dequeue, if `stack2` is empty, pour all of `stack1` into `stack2` and pop from `stack2` (amortized $O(1)$ per operation).
- **Priority queue stability:** a standard binary heap does NOT guarantee FIFO order among equal-priority elements; if needed, pair `(priority, insertion_index)` and compare both.
- **Deque vs restricted deque:** a plain deque allows insertion/deletion at both ends; restricted deques (Section 7 & 8) are the same underlying structure with the public interface deliberately narrowed.

12. Quick Reference — Time Complexity Table

Operation	Linear Queue (array)	Circular Queue	Linked Queue	Deque	Priority Queue (heap)
Enqueue/Insert	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Dequeue/Delete	$O(1)^*$	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Peek (front/rear)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

* $O(1)$ time but causes unusable wasted space over repeated cycles (see Section 4).